EXE-2

Step-0 Data Processing

Python libraries such as pandas, matplotlib, and seaborn were used to handle and visualize the dataset. Pandas was used for loading, inspecting, and cleaning the data, while matplotlib and seaborn were used to generate plots that help analyze relationships between variables and understand patterns in the dataset.

Step-1 Problem Defination

The objective of this experiment is to build a machine learning model that can predict house prices using the given dataset. The dataset contains features such as area, number of bedrooms, location, and other property attributes. Since price is a continuous numeric value, this problem is a regression task. The model must learn how these features influence price and estimate it accurately.

Step-2 Data understanding

Using functions like df.head() and df.info(), the dataset structure was examined. Each row represents a house, and each column represents a feature describing that house.

The dataset contains:

Numerical features : Area, Bedrooms, Bathrooms, Age, Parking, Balcony, etc.

Categorical features : Location and Furnished

The target variable is Price, which is denoted as y, and the remaining features are used as input variables x. Since machine learning models require numeric input, categorical features must be encoded before training.

Step-3 Data Processing

Categorical variables such as Location and Furnished were converted into numerical format using one-hot encoding. This creates new columns such as:

- Location_Urban
- Location_Rural
- Furnished_Yes

This transformation was done using the get_dummies() function. Numerical features were already in usable format and therefore did not require modification. This preprocessing step ensures the dataset is suitable for machine learning algorithms. This aligns with the preprocessing stage of the pipeline where raw data is transformed into model-ready numerical form

Step-4 Data Selection and Model Training

As discussed earlier, Price was selected as the target variable (y), and all other columns were selected as input features (x). For demonstration purposes, the model was trained on the full dataset instead of splitting into training and testing sets.

The model chosen was Linear Regression, since the dependent variable is continuous. Regression models are specifically designed to predict numeric outputs, unlike classification models which predict categories.

Step 5-Prediction

After training, the model was used to generate predictions using the .predict() function. By providing input feature values (X), the model calculates estimated house prices based

on the learned relationship between features and target variable.

Step-6 Evaluation

The model performance was evaluated using:

Mean Squared Error (MSE) -- measures average squared difference between predicted and actual prices

R^2 Score - measures how well the model explains variation in price .

The obtained values showed MSE close to zero and R^2 close to 1, indicating that the model fit the training data extremely well. According to regression evaluation theory, lower MSE

Additionally, the results were visualized using a scatter plot to compare predicted prices with actual prices, allowing visual inspection of model accuracy.

Predicting Housing Prices: Develop a regression model to predict house prices based on features like location, size, and amenities. italicized text

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn import linear_model as lm
from sklearn.metrics import mean_squared_error, r2_score
```

```
df=pd.read_csv('/content/drive/MyDrive/Housing_Price_Dataset_120_Records.csv')
print(df.head())
```

```
  HouseID  Location  Area_sqft  Bedrooms  Bathrooms  Balcony  Parking  \
0    H001     Rural       1028         1          3        0        0
1    H002     Rural       2316         1          4        0        0
2    H003     Urban       1834         1          2        2        2
3    H004     Urban       1719         3          1        0        2
4    H005  Suburban       1118         2          3        0        0

   Age_Years Furnished   Price
0          5       Yes   55.84
1          3       Yes  103.88
2         23        No   86.42
3         14        No   96.77
4         13       Yes   63.94
```

Data Preprocessing

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 120 entries, 0 to 119
Data columns (total 10 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   HouseID    120 non-null    object
 1   Location   120 non-null    object
 2   Area_sqft  120 non-null    int64
 3   Bedrooms   120 non-null    int64
 4   Bathrooms  120 non-null    int64
 5   Balcony    120 non-null    int64
 6   Parking    120 non-null    int64
 7   Age_Years  120 non-null    int64
 8   Furnished  120 non-null    object
 9   Price      120 non-null    float64
dtypes: float64(1), int64(6), object(3)
memory usage: 9.5+ KB
```

```
pd.Index(['HouseID','Location','Area_sqft','Bedrooms','Bathrooms','Balcony','Parking','Age_Years','Furnished','Price'])
```

```
Index(['HouseID', 'Location', 'Area_sqft', 'Bedrooms', 'Bathrooms', 'Balcony',
       'Parking', 'Age_Years', 'Furnished', 'Price'],
      dtype='object')
```

```
df=pd.get_dummies(df,columns=['Furnished','Location'],drop_first=False)
```

```
df.head()
```

	HouseID	Area_sqft	Bedrooms	Bathrooms	Balcony	Parking	Age_Years	Price	Furnished_No	Furnished_Yes	Location_Rural
0	H001	1028	1	3	0	0	5	55.84	False	True	True
1	H002	2316	1	4	0	0	3	103.88	False	True	True
2	H003	1834	1	2	2	2	23	86.42	True	False	False
3	H004	1719	3	1	0	2	14	96.77	True	False	False
4	H005	1118	2	3	0	0	13	63.94	False	True	False

```
reg=lm.LinearRegression()
reg.fit(df[['Area_sqft','Bedrooms','Bathrooms','Balcony','Parking','Age_Years','Furnished_Yes','Location_Suburban','Locatio
```

```
▾ LinearRegression
LinearRegression()
```

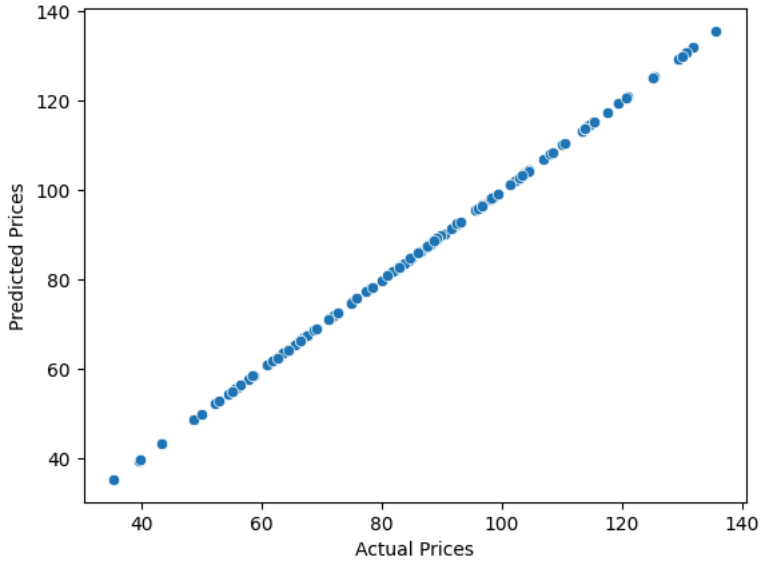
```
import pickle
with open('model_pickle','wb') as f:
  pickle.dump(reg,f)
```

```
y_pred = reg.predict(df[['Area_sqft','Bedrooms','Bathrooms','Balcony','Parking','Age_Years','Furnished_Yes','Location_Subur
mse = mean_squared_error(df.Price, y_pred)
r2 = r2_score(df.Price, y_pred)
print(f'Mean Squared Error: {mse}')
print(f'R^2 Score: {r2}')
```

```
Mean Squared Error: 6.344966032873642e-27
R^2 Score: 1.0
```

```
plot = sns.scatterplot(x=df.Price, y=y_pred)
plot.set_xlabel('Actual Prices')
plot.set_ylabel('Predicted Prices')
```

```
Text(0, 0.5, 'Predicted Prices')
```



EXE-3

Step-0 Libraries Used

Several Python libraries were used to perform data handling, model training, evaluation, and visualization for stock price prediction.

NumPy: was used for numerical operations and array manipulation required during model input preparation.

Pandas: was used for loading the dataset, filtering stock data, converting date formats, and creating new features such as the numerical time variable (Days).

Matplotlib and **Seaborn:** were used for visualization to plot stock price trends and analyze the distribution of values.

Scikit-learn: was used to implement machine learning functionality. The train_test_split module was used to divide data into training and testing sets, the LinearRegression model was used for prediction, and evaluation metrics such as **Mean Squared Error** and **R^2 score** were used to assess model performance.

Pickle: was used to save the trained model so it can be reused later without retraining.

Step-1 Problem Definition

The objective of this experiment is to develop a time series prediction model to forecast stock prices. The dataset contains historical stock data including Date, Open, High, Low, Close prices, and Volume for different companies. Since the output variable (stock price) is numeric and changes over time, this is a regression-based time series prediction problem.

Step-2 Data Understanding

The dataset was loaded using pandas and inspected to understand its structure. Each row represents a stock record for a specific date, and each column represents a feature describing that day's market performance.

The dataset contains the following attributes:

- Date → trading day
- Stock → company ticker symbol
- Open, High, Low, Close → price values
- Volume → number of shares traded

As shown in the dataset, the data contains multiple stocks such as AAPL and TSLA ,For modeling purposes, only one stock (AAPL) was selected so that a single time series could be analyzed.

Step-3 Data Preprocessing

The Date column was converted into datetime format to enable time-based calculations. A new feature called Days was created by subtracting the minimum date from each date value. This converts time into numerical form representing the number of days since the start of the dataset.

This step is necessary because machine learning models cannot directly process date values; they require numerical inputs.

Step-4 Feature Selection, Model Selection and Training

In this step, the input (X) was taken as the Days column, which represents time in numeric form, and the output (y) was selected as the closing stock price.

The dataset was divided into training and testing sets using an 80:20 ratio. Shuffling was disabled so that the chronological

order of dates remains unchanged. This is important for time-series data because predictions should always be made using past values to estimate future ones.

A Linear Regression model was chosen for this task because the target variable is continuous. The model was trained using the training data, where it learned the relationship between time and stock price by fitting a line that minimizes prediction error. After training, the model parameters were stored so that it could later be reused without retraining.

Step-6 Prediction

After training, the model was used to predict stock prices for the test dataset. The predictions generated represent estimated closing prices based solely on time progression.

As shown in the output, the predicted values form a continuous decreasing trend, indicating the model learned a linear relationship

Step-7 Evaluation

Model performance was evaluated using two regression metrics:

- Mean Squared Error (MSE):Measures average squared difference between predicted and actual values. Lower values indicate better performance.

- R^2 Score:Measures how well the model explains variance in the data. Values closer to 1 indicate better fit.

From the results :

MSE is approx.= 3072

R^2 is approx.= -11.21

The negative R^2 indicates the model performs poorly and does not fit the data well. This occurs because stock prices follow nonlinear patterns, while linear regression can only model straight-line relationships.

Predicting Stock Prices: Develop a time series prediction model to forecast stock prices.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn import linear_model as lm
from sklearn.metrics import r2_score,mean_squared_error
import pickle
```

```
df=pd.read_csv('/content/drive/MyDrive/stock_price_timeseries_dataset.csv')
df
```

	Date	Stock	Open	High	Low	Close	Volume
0	2020-01-01	AAPL	173.80	174.75	173.68	174.43	8204212
1	2020-01-02	AAPL	176.01	177.59	174.07	177.55	2766891
2	2020-01-03	AAPL	177.57	178.37	176.62	176.71	5721339
3	2020-01-06	AAPL	176.01	177.58	171.33	171.73	9242680
4	2020-01-07	AAPL	171.85	172.20	170.69	170.82	4416664
...	...	...	...	...	...	...	...
6520	2024-12-25	TSLA	225.09	225.21	221.66	223.18	9701165
6521	2024-12-26	TSLA	222.07	222.87	220.31	220.32	6045143
6522	2024-12-27	TSLA	219.94	221.69	218.38	219.57	8512677
6523	2024-12-30	TSLA	219.54	220.33	216.70	217.33	9137479
6524	2024-12-31	TSLA	216.47	219.61	216.19	218.41	2097124

6525 rows × 7 columns

Next steps: Generate code with df New interactive sheet

```
df_stock=df[df['Stock']=='AAPL']
df_stock.head()
```

	Date	Stock	Open	High	Low	Close	Volume
0	2020-01-01	AAPL	173.80	174.75	173.68	174.43	8204212
1	2020-01-02	AAPL	176.01	177.59	174.07	177.55	2766891
2	2020-01-03	AAPL	177.57	178.37	176.62	176.71	5721339
3	2020-01-06	AAPL	176.01	177.58	171.33	171.73	9242680
4	2020-01-07	AAPL	171.85	172.20	170.69	170.82	4416664

Next steps: Generate code with df_stock New interactive sheet

```
df_stock['Date']=pd.to_datetime(df_stock['Date'])
df_stock.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1305 entries, 0 to 1304
Data columns (total 7 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Date    1305 non-null   datetime64[ns]
 1   Stock   1305 non-null   object
 2   Open    1305 non-null   float64
 3   High    1305 non-null   float64
 4   Low     1305 non-null   float64
 5   Close   1305 non-null   float64
 6   Volume  1305 non-null   int64
dtypes: datetime64[ns](1), float64(4), int64(1), object(1)
memory usage: 113.9+ KB
```

```
/tmp/ipython-input-31740583.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
  df_stock['Date']=pd.to_datetime(df_stock['Date'])
```

```
df_stock['Days']=(df_stock['Date']-df_stock['Date'].min())
df_stock
```

```
/tmp/ipython-input-1482232864.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
  df_stock['Days']=(df_stock['Date']-df_stock['Date'].min())
```

	Date	Stock	Open	High	Low	Close	Volume	Days
0	2020-01-01	AAPL	173.80	174.75	173.68	174.43	8204212	0 days
1	2020-01-02	AAPL	176.01	177.59	174.07	177.55	2766891	1 days
2	2020-01-03	AAPL	177.57	178.37	176.62	176.71	5721339	2 days
3	2020-01-06	AAPL	176.01	177.58	171.33	171.73	9242680	5 days
4	2020-01-07	AAPL	171.85	172.20	170.69	170.82	4416664	6 days
...	...	...	...	...	...	...	...	...
1300	2024-12-25	AAPL	207.96	210.35	206.18	208.42	4510463	1820 days
1301	2024-12-26	AAPL	208.69	214.63	207.23	213.99	4335705	1821 days
1302	2024-12-27	AAPL	213.99	217.12	212.77	216.24	2601207	1822 days
1303	2024-12-30	AAPL	216.62	217.17	213.72	215.53	7203690	1825 days
1304	2024-12-31	AAPL	215.26	218.50	215.06	217.00	5083372	1826 days

1305 rows × 8 columns

Next steps: Generate code with df_stock New interactive sheet

```
x=df_stock['Days']
y=df_stock['Close']
```

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,shuffle=False)
```

```
model=lm.LinearRegression()
model.fit(x_train.dt.days.values.reshape(-1,1), y_train)
with open('model_pickle','wb') as f:
  pickle.dump(model,f)
```

```
y_pred=model.predict(x_test.dt.days.values.reshape(-1,1))
y_pred
```

```
       150.49573228, 150.47262862, 150.40331765, 150.380214  ,
       150.35711034, 150.33400669, 150.31090303, 150.24159206,
       150.21848841, 150.19538475, 150.17228109, 150.14917744,
       150.07986647, 150.05676281, 150.03365916, 150.0105555 ,
       149.98745185, 149.91814088, 149.89503722, 149.87193357,
       149.84882991, 149.82572625, 149.75641529, 149.73331163,
       149.71020797, 149.68710432, 149.66400066, 149.59468969,
       149.57158604, 149.54848238, 149.52537873, 149.50227507,
       149.4329641 , 149.40986045, 149.38675679, 149.36365313,
       149.34054948, 149.27123851, 149.24813485, 149.2250312 ,
       149.20192754, 149.17882389, 149.10951292, 149.08640926,
       149.06330561, 149.04020195, 149.01709829, 148.94778733,
       148.92468367, 148.90158001, 148.87847636, 148.8553727 ,
       148.78606173, 148.76295808, 148.73985442, 148.71675077,
       148.69364711, 148.62433614, 148.60123249, 148.57812883,
       148.55502517, 148.53192152, 148.46261055, 148.43950689,
       148.41640324, 148.39329958, 148.37019593, 148.30088496,
       148.2777813 , 148.25467765, 148.23157399, 148.20847033,
       148.13915937, 148.11605571, 148.09295205, 148.0698484 ,
       148.04674474, 147.97743377, 147.95433012, 147.93122646,
```

```
       146.84353463, 146.82223097, 146.79914752, 146.77604566,
       146.75294001, 146.68362904, 146.66052538, 146.63742173,
       146.61431807, 146.59121441, 146.52190345, 146.49879979,
       146.47569613, 146.45259248, 146.42948882, 146.36017785,
       146.3370742 , 146.31397054, 146.29086689, 146.26776323,
       146.19845226, 146.17534861, 146.15224495, 146.12914129,
       146.10603764, 146.03672667, 146.01362301, 145.99051936,
       145.9674157 , 145.94431205, 145.87500108, 145.85189742,
       145.82879377, 145.80569011, 145.78258645, 145.71327549,
       145.69017183, 145.66706817, 145.64396452, 145.62086086,
       145.55154989, 145.52844624, 145.50534258, 145.48223893,
       145.45913527, 145.3898243 , 145.36672065, 145.34361699,
       145.32051333, 145.29740968, 145.22809871, 145.20499505,
       145.1818914 , 145.15878774, 145.13568409, 145.06637312,
       145.04326946, 145.02016581, 144.99706215, 144.97395849,
       144.90464753, 144.88154387, 144.85844021, 144.83533656,
       144.8122329 , 144.74292193, 144.71981828, 144.69671462,
       144.67361097, 144.65050731, 144.58119634, 144.55809269,
       144.53498903, 144.51188537, 144.48878172, 144.41947075,
       144.39636709, 144.37326344, 144.35015978, 144.32705613,
       144.25774516, 144.2346415 , 144.21153785, 144.18843419,
       144.16533053, 144.09601957, 144.07291591, 144.04981225,
       144.0267086 , 144.00360494, 143.93429397, 143.91119032,
       143.88808666, 143.86498301, 143.84187935, 143.77256838,
       143.74946473, 143.72636107, 143.70325741, 143.68015376,
       143.61084279, 143.58773913, 143.56463548, 143.54153182,
       143.51842817, 143.4491172 , 143.42601354, 143.40290989,
       143.37980623, 143.35670257, 143.28739161, 143.26428795,
       143.24118429, 143.21808064, 143.19497698, 143.12566601,
       143.10256236])
```

```
mse=mean_squared_error(y_test,y_pred)
mse
```

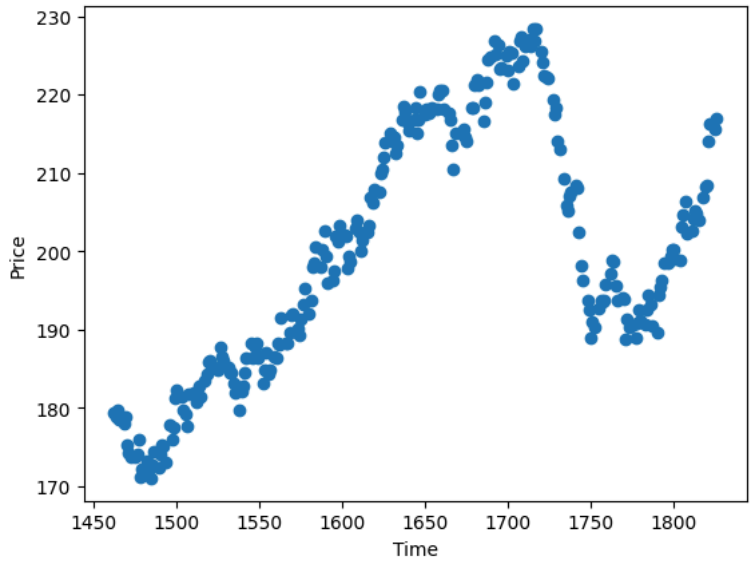
```
3072.378496877529
```

```
r2=r2_score(y_test,y_pred)
r2
```

```
-11.213999810715315
```

```
plt.scatter(x_test.dt.days,y_test)
plt.xlabel('Time')
plt.ylabel('Price')
```

```
Text(0, 0.5, 'Price')
```



EX-4

Step-0 Libraries Used

Pandas: is used to load and analyze data.

NumPy: is used for numerical operations.

Matplotlib and Seaborn: are used for plotting graphs.

Scikit-learn: is used for train test split, model training, prediction, and evaluation.

Pickle: is used to save the trained model for later use.

Step-1 Problem Definition

The goal is to predict whether a customer will churn or not using customer information such as age, subscription type, payment method, and charges. Since the output is either 0 or 1, this is a binary classification problem.

Step-2 Data Understanding

The dataset was loaded and checked using head() and info() to understand structure. Each row represents a customer and each column represents a feature.

The dataset contains numerical features such as age, tenure, charges, and support tickets, and categorical features such as gender, subscription type, payment method, and contract type. The target column is Churn.

Step-3 Data Processing

The CustomerID column was removed because it is only an identifier and does not help prediction. Categorical columns were converted into numerical form using one-hot encoding so that the model can understand them.

Step-4 Feature Selection and Model Training

All columns except Churn were selected as input (x), and Churn was selected as output (y). The dataset was split into training and testing sets in 80:20 ratio. A Logistic Regression model was chosen because it is suitable for binary classification. The model was trained on training data to learn patterns between customer features and churn.

Step-5 Prediction

After training, predictions were generated using the test data. The model outputs either 0 (not churn) or 1 (churn) based on learned patterns.

Step-6 Evaluation

The model performance was evaluated using accuracy, precision, recall, and F1 score. The accuracy was about 0.75, which shows the model performs reasonably well. However, precision and recall were lower, meaning the model has difficulty correctly identifying churn customers.

A confusion matrix was plotted to visualize prediction performance and to see correct and incorrect classifications.

Step-7 Feature Impact

A graph of model coefficients was plotted to see which features influence churn prediction the most. Features related to subscription type, payment method, and contract duration showed strong impact.

Customer Churn Prediction: Develop a model to predict customer churn in a subscription-based business.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score,confusion_matrix
import pickle
```

```
df=pd.read_csv('/content/drive/MyDrive/customer_churn_dataset.csv')
df.head()
```

	CustomerID	Gender	Age	Tenure_Months	SubscriptionType	MonthlyCharges	TotalCharges	PaymentMethod	SupportTickets	Co
0	CUST1000	Male	49	54	Standard	99.33	4912.77	Net Banking	1	
1	CUST1001	Female	56	8	Standard	12.01	604.59	Debit Card	2	
2	CUST1002	Male	66	27	Basic	57.62	2019.39	Cash	4	
3	CUST1003	Male	69	27	Premium	97.08	4848.88	Credit Card	7	
4	CUST1004	Male	49	34	Standard	54.69	4334.26	Credit Card	9	

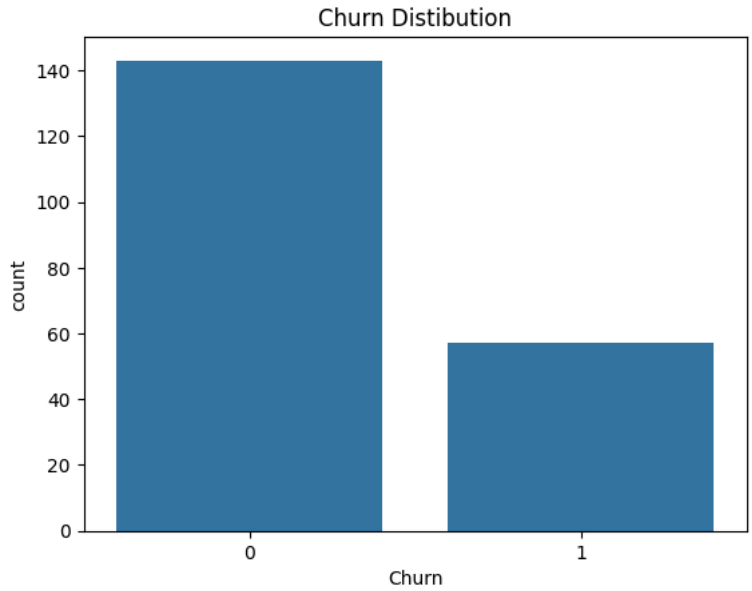
Next steps: Generate code with df | New interactive sheet

```
print("Shape:",df.shape)
df.info()
df.describe()
```

```
Shape: (200, 11)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 11 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   CustomerID        200 non-null    object
 1   Gender            200 non-null    object
 2   Age               200 non-null    int64
 3   Tenure_Months     200 non-null    int64
 4   SubscriptionType  200 non-null    object
 5   MonthlyCharges    200 non-null    float64
 6   TotalCharges      200 non-null    float64
 7   PaymentMethod     200 non-null    object
 8   SupportTickets    200 non-null    int64
 9   ContractType      200 non-null    object
 10  Churn             200 non-null    int64
dtypes: float64(2), int64(4), object(5)
memory usage: 17.3+ KB
```

	Age	Tenure_Months	MonthlyCharges	TotalCharges	SupportTickets	Churn
count	200.000000	200.000000	200.000000	200.000000	200.000000	200.000000
mean	44.405000	31.925000	51.348250	2536.105100	4.505000	0.285000
std	15.353816	16.390933	27.570371	1408.107184	2.967153	0.452547
min	18.000000	1.000000	6.380000	72.930000	0.000000	0.000000
25%	32.750000	21.000000	27.335000	1292.312500	2.000000	0.000000
50%	46.000000	32.000000	50.530000	2537.170000	4.000000	0.000000
75%	56.000000	46.250000	73.062500	3753.577500	7.000000	1.000000
max	69.000000	58.000000	99.970000	4984.530000	9.000000	1.000000

```
sns.countplot(x='Churn',data=df)
plt.title("Churn Distibution")
plt.show()
```




```
df=df.drop("CustomerID",axis=1)
df=pd.get_dummies(df,drop_first=True)
df.head()
```

	Age	Tenure_Months	MonthlyCharges	TotalCharges	SupportTickets	Churn	Gender_Male	SubscriptionType_Premium	Subscript
0	49	54	99.33	4912.77	1	0	True	False	
1	56	8	12.01	604.59	2	0	False	False	
2	66	27	57.62	2019.39	4	0	True	False	
3	69	27	97.08	4848.88	7	0	True	True	
4	49	34	54.69	4334.26	9	1	True	False	

Next steps: Generate code with df New interactive sheet

```
X=df.drop(["Churn"],axis=1)
y=df["Churn"]
X.head(),y.head()
```

```
(   Age  Tenure_Months  MonthlyCharges  TotalCharges  SupportTickets  \
 0   49            54           99.33       4912.77               1
 1   56             8           12.01        604.59               2
 2   66            27           57.62       2019.39               4
 3   69            27           97.08       4848.88               7
 4   49            34           54.69       4334.26               9

    Gender_Male  SubscriptionType_Premium  SubscriptionType_Standard  \
 0         True                     False                       True
 1        False                     False                       True
 2         True                     False                      False
 3         True                      True                      False
 4         True                     False                       True

    PaymentMethod_Credit Card  PaymentMethod_Debit Card  \
 0                      False                     False
 1                      False                      True
 2                      False                     False
 3                       True                     False
 4                       True                     False

    PaymentMethod_Net Banking  PaymentMethod_UPI  ContractType_One Year  \
 0                       True              False                  False
 1                      False              False                  False
 2                      False              False                  False
 3                      False              False                  False
 4                      False              False                   True

    ContractType_Two Year
 0                   True
 1                   True
 2                   True
 3                   True
 4                  False  ,
 0    0
```

```
1    0
2    0
3    0
4    1
Name: Churn, dtype: int64)
```

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=50)
print("Train size:",X_train.shape)
print("Test size:",X_test.shape)
```

```
Train size: (160, 14)
Test size: (40, 14)
```

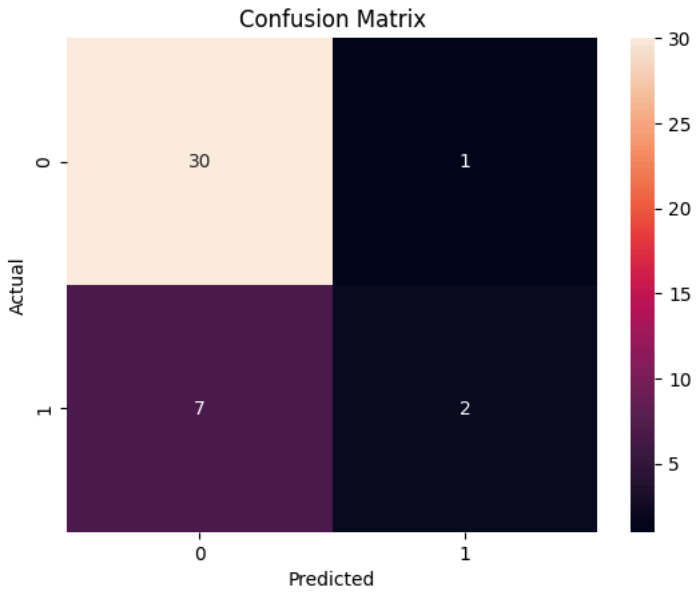
```
model=LogisticRegression(max_iter=1000)
model.fit(X_train,y_train)
with open("model","wb") as f:
    pickle.dump(model,f)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

```
y_pred=model.predict(X_test)
y_pred
```

```
array([0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0])
```

```
cm=confusion_matrix(y_test,y_pred)

sns.heatmap(cm,annot=True,fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```




```
print("Accuracy:",accuracy_score(y_test,y_pred))
print("Precision:",precision_score(y_test,y_pred))
print("Recall:",recall_score(y_test,y_pred))
print("F1 Score:",f1_score(y_test,y_pred))
```

```
Accuracy: 0.8
Precision: 0.6666666666666666
Recall: 0.2222222222222222
F1 Score: 0.3333333333333333
```